

samy kamkar - NAT Pinning

samy kamkar - NAT Pinning - Author not stated, samy.pl.

- Published: date not stated
- Original: <<http://samy.pl/natpin/>>
- Preserved from: <http://samy.pl/natpin/> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

samy kamkar - NAT Pinning

[home page](#) || [follow my twitter](#) || [blog](#) || [email me](#) || [samy kamkar](#)

NAT Pinning: Penetrating routers and firewalls from a web page (forcing router to port forward)

Welcome. Here is a proof of concept in what I'm calling **NAT Pinning** ("hacking gibsons" was already taken). The idea is an attacker lures a victim to a web page. The web page forces the user's router or firewall, unbeknownst to them, to port forward any port number back to the user's machine. If the user had FTP/ssh/etc open but it was blocked from the router, it can now be forwarded for anyone to access (read: attack) from the outside world. No XSS or CSRF required.

My method works like this:

1. Attacker lures victim to a URL by convincing them that there are pictures of cute kittens on the page.
2. Victim clicks on URL and opens the page.
3. The page has a hidden form connecting to <http://attacker.com:6667> ([IRC](#) port).
4. The client (victim) submits the form without knowing. An HTTP connection is created to the (fake) IRC server.
5. The fake IRC server, run by the attacker, simply listens, unlike me according to former girlfriends.
6. The form also has a hidden value that sends: "PRIVMSG samy :\1DCC CHAT samy [ip in decimal] [port]\1\n"
7. Your router, doing you a favor, sees an "IRC connection" (even though your client is speaking in HTTP) and an attempt at a "[DCC chat](#)". DCC chats require opening a local port on the client for the remote chatter to connect back to you.
8. Since the router is blocking all inbound connections, it decides to forward any traffic to the port in the "DCC chat" back to you to allow NAT traversal for the friendly attacker to connect back and

"chat" with you. However, the attacker specified the port to be, for example, port 21 (FTP). The router port forwards 21 back to the victim's internal system. The attacker now has a clear route to connect to the victim on port 21 and launch an attack, downloading the victim's highly classified cute kitten pictures.

Want to test? After you click the button below, try **telnet 86.133.176.148 [port]** on a system that is not on your network. Port:

Not all routers support this method of NAT traversal -- using the FTP method is far superior. I

chose IRC in this example because IRC connection tracking support is in older versions of Linux, some routers' FTP's connection tracking only works on inbound connections, and IRC is just way more fun. I've tested this successfully on a Belkin N1 Vision Wireless Router and worked out of the box (the IRC method failed on a Netopia 3347-02).

To use FTP, you'll just need to send a "227 samy was here (192,168,0,1,20,30)\n", however it needs to be on port 21 and on some routers must be on an *inbound* connection. You'll want to use an attack [like this (FF/Opera only)]() to get their internal IP. In this scenario, the internal IP is 192.168.0.1 and the port to connect to is 5150 (20 = 0x14, 30 = 0x1e, 0x141e = 5150).

To view other cool stuff, check out [my website](#) or [follow my twitter](#).

developed by [samy kamkar](#), 01/05/2010